DSA03-NATURAL LANGUAGE PROCESSING ASSIGNMENT-1

ASSIGNMENT 1 – CO1

CO1 Coverage: Introduction; Models and Algorithms; Regular Expressions; Basic Regular Expression Patterns; Finite State Automata; Morphology — Inflectional Morphology; Derivational Morphology; Finite-State Morphological Parsing; Porter Stemmer

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Assignment Title: Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System Using Regular Expressions, Finite-State Automata and Morphological Parsing (Total Marks – 50)

Duration: 1 week

Scenario — Text Normalisation and Search for a Hospital Record System

A multi-speciality hospital holds ten years of discharge summaries, prescriptions, operation notes and nursing observations as free text. Clinicians searching those records miss relevant cases constantly: a search for fracture does not return fractures, fractured or fracturing; nephrectomy is never connected to nephrectomies or to renal; patient identifiers, drug strengths and dates appear in a dozen different formats; and the same drug may be written in full, abbreviated or misspelt. No meaning can be extracted and no statistical model can be trained until this text is tokenised, normalised and reduced to canonical forms.

You are the language-engineering team. Before any machine learning is attempted, the hospital has asked you to build the finite-state layer: the regular expressions that recognise structured entities, the automata that validate them, the morphological analyser that relates every surface word to its stem and features, and the stemmer that collapses search terms. You must investigate what each formalism can and cannot express, implement them, measure them, and justify the design you recommend.

Planning data and constraints

- Corpus: at least 500 de-identified clinical records (real or synthetic, but documented), totalling at least 25,000 word tokens and covering at least four record types.
- Pattern inventory: at least 12 structured entity types to be recognised by regular expression — patient identifier, admission and discharge date, drug name with

strength, dosage frequency, laboratory value with unit, blood pressure, temperature, ICD code, ward and bed, contact number, age, and duration of symptom.

- Automata: at least four finite-state automata, each with at least six states, including one entity format that cannot be recognised conveniently without non-determinism.
- Morphological resources (build and document your own): a lexicon of at least 300 stems, at least 20 inflectional affixes and at least 15 derivational affixes, with irregular forms listed separately.
- Evaluation data: at least 40 clinician queries with relevance judgements over the corpus, and at least 200 word forms hand-annotated with their stem and features.
- Any manual effort (pattern writing, lexicon building, annotation, testing) must be accounted for in your effort and cost discussion.

Design Challenge (Day 4):

The system must now ingest notes captured by optical character recognition from scanned handwritten forms, and notes typed in Romanised regional language mixed with English. Re-evaluate your original patterns, automata and analyser and decide whether to retain, modify or redesign them.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis 3 Marks

Guiding Enquiry

Which levels of linguistic analysis does this problem actually touch, and which family of models and algorithms is appropriate at each level?

Investigate and interpret:

- Describe the levels of linguistic analysis — phonology, morphology, syntax, semantics, pragmatics and discourse — and state, with an example drawn from your own corpus, which levels the record-search problem requires and which it can safely ignore.
- Distinguish the three model families used throughout this unit: state machines, formal rule systems and probabilistic models. Name one task in the hospital problem that each family suits best and say why the other two would be the wrong choice for it.
- Explain why ambiguity is the central problem of natural language processing, and identify at least four distinct ambiguities present in your corpus — lexical, morphological, segmentation and abbreviation ambiguity.
- Identify where search appears in this unit (state-space search in automata, backtracking in matching) and state the resources each consumes: time, memory, annotated data and expert effort.
- Set out, as a labelled diagram, the pipeline you will build in the remaining tasks, naming the input, the output and the formalism of every stage.

Evidence Expected

A reasoned problem framing that maps the hospital task onto levels of analysis, model families and a concrete, staged pipeline.

TASK 2 – Basic Regular Expression Patterns 4 Marks

Guiding Enquiry

What can the basic regular expression operators express, and how are they combined into patterns that match real clinical strings exactly?

Investigate and interpret:

- Define the basic operators — concatenation, disjunction, Kleene star and plus, optionality, character classes, ranges, negated classes, anchors and counters — and give one minimal clinical example of each.
- Write and document a regular expression for each of the twelve entity types in your pattern inventory, stating for every pattern both what it must match and what it must not.
- Demonstrate greedy and non-greedy matching on a clinical sentence containing two dosage expressions, and explain precisely why the two quantifiers return different spans.
- Explain operator precedence and grouping, and show with a worked example how a misplaced disjunction silently changes the language the expression denotes.
- Evaluate every pattern over the corpus, reporting true positives, false positives and false negatives, and name the specific linguistic cause of each error class.

Evidence Expected

A documented pattern inventory with precise match specifications and a per-pattern error analysis carried out on real data.

TASK 3 – Regular Expressions for Tokenisation and Normalisation 4 Marks

Guiding Enquiry

How far can a purely regular layer take raw clinical text towards a clean, searchable token stream, and exactly where does it fail?

Investigate and interpret:

- Specify your tokenizer as a single regular expression, defining precisely what counts as a token — hyphenated terms, abbreviations containing full stops, decimal values, units written against numbers — and justify each decision.
- Apply maximal-munch (longest-match) tokenisation and show on at least three examples how the result differs from a naive split on whitespace.
- Build substitution rules that fold case, expand at least thirty clinical abbreviations and canonicalise date and unit formats; show the complete before-and-after text of one full record.
- Identify at least three normalisation decisions that are irreversible because information is destroyed, and argue when that loss is acceptable and when it is not.
- Give one requirement of the hospital task that no regular expression can express, and argue informally why — for example, matching balanced parentheses in a nested clinical note.

Evidence Expected

A working regular-expression tokenizer and normaliser with worked examples and a clear statement of the limits of regularity.

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence 5 Marks

Guiding Enquiry

How do automata recognise exactly the same languages as regular expressions, and what does removing non-determinism cost?

Investigate and interpret:

- Define a finite automaton formally as the five-tuple $(Q, \Sigma, \delta, q_0, F)$, then design a deterministic automaton for at least four validators from your pattern inventory, giving both the transition diagram and the transition table for each.
- Design a non-deterministic automaton with ε-transitions for one entity whose format has optional parts, and trace its behaviour on one accepted and one rejected string, showing the set of states live at every step.
- Convert that NFA to a DFA by subset construction, showing the complete construction table, then minimise the DFA and report the state counts before and after.
- Apply Thompson's construction to turn one Task 2 regular expression into an automaton, and derive a regular expression back from one of your automata, demonstrating the equivalence in both directions.
- State the pumping lemma for regular languages and use it to prove that one specific clinical pattern is not regular — for example a note requiring equal numbers of opening and closing brackets — and relate the proof to your finding in Task 3.

Evidence Expected

Correct automata with full subset-construction and minimisation tables, a demonstrated two-way equivalence with regular expressions, and a valid non-regularity proof.

TASK 5 – Inflectional Morphology 3 Marks

Guiding Enquiry

Which inflectional processes appear in clinical English, and how are surface forms systematically related to their stems?

Investigate and interpret:

- Define morpheme, stem, affix, inflection and paradigm, and separate inflection from derivation using the standard criteria of category preservation, productivity and position relative to other affixes.
- List every inflectional paradigm present in your corpus — noun number, verb tense and aspect, subject agreement, adjective comparison — and give the full paradigm for at least five clinical lexemes.
- State the orthographic rules governing English inflection — e-deletion, consonant doubling, y-to-i, the sibilant -es rule — precisely enough to be implemented, and give for each one a counter-example on which it over-generates.
- Identify the irregular forms in your corpus and explain why each must be listed in the lexicon rather than derived by rule.
- Measure the effect of inflection: report the type-to-lemma ratio of your corpus and the proportion of failures in your query set that inflection alone explains.

Evidence Expected

Precisely stated inflectional rules with paradigms, an irregular list, and a quantified measurement of the effect of inflection on retrieval.

TASK 6 – Derivational Morphology 4 Marks

Guiding Enquiry

How does derivation build the specialised clinical vocabulary, and why is it harder to handle than inflection?

Investigate and interpret:

- Define derivation and list at least fifteen derivational affixes attested in your corpus — for example -itis, -ectomy, -ology, -osis, -aemia, -ation, -able, -ive, nephro-, hyper-, hypo-, re-, un- — stating the category change each one causes.
- Build the derivational family of at least six clinical roots, showing the order in which affixes attach and identifying at least two orders that are impossible.
- Explain semantic non-compositionality with clinical examples in which the meaning of the derived word cannot be predicted from its parts, and state what this implies for a search system that stems aggressively.
- Compare the productivity of your affixes empirically: count types and tokens for each in the corpus and classify each affix as productive or fossilised, with the evidence for the classification.
- Decide whether derivational relatives should be conflated for search, citing at least two cases where conflation helps the clinician and two where it produces a clinically misleading match.

Evidence Expected

A documented derivational analysis with an affix inventory, productivity measurements and a reasoned position on conflation.

TASK 7 – Finite-State Morphological Parsing 5 Marks

Guiding Enquiry

How can a transducer relate a surface form to its stem and features, and run in both directions on the same machine?

Investigate and interpret:

- Define a finite-state transducer as a two-tape automaton and explain the lexical and surface tapes with one fully worked clinical example.
- Build your lexicon as an FST organised into continuation classes, covering at least three parts of speech together with the inflectional and derivational affixes of Tasks 5 and 6.
- Hand-trace the parsing of at least five surface forms, showing the state sequence, the symbols consumed and emitted on each tape, and the final analysis — for example fractures → fracture +N +PL.
- Run the same transducer in the generation direction, producing surface forms from stem-plus-feature input, and show explicitly that parsing and generation use one machine and not two.
- Handle ambiguity: give at least two surface forms with more than one analysis, show the transducer returning all of them, and state how a downstream component would choose between them.

Evidence Expected

A working transducer specification with traced analyses in both directions and an explicit treatment of ambiguity.

TASK 8 – Orthographic Rules and Two-Level Morphology 4 Marks

Guiding Enquiry

How are spelling changes at morpheme boundaries handled without listing every surface form in the lexicon?

Investigate and interpret:

- Explain why a lexicon transducer alone cannot produce correct surface spellings, using at least three examples in which plain concatenation yields the wrong string.
- Write at least five orthographic rules in the standard rewrite notation (a → b / c __ d), stating the context of each rule precisely.
- Implement each rule as a transducer and compose the rule transducers with the lexicon FST; draw the cascade and state the order dependencies between the rules.
- Contrast this sequential cascade with the parallel two-level (Koskenniemi) formulation, and explain the practical advantage each has over the other.
- Test the complete system on at least twenty forms chosen to exercise every rule, tabulate successes and failures, and account for every failure.

Evidence Expected

Correct rule transducers, a documented composition order, and a tested cascade with full error accounting.

TASK 9 – The Porter Stemmer 5 Marks

Guiding Enquiry

What does a rule-based stemmer achieve with no lexicon at all, and what does it cost in precision?

Investigate and interpret:

- State the algorithm precisely: the definition of consonant and vowel, the measure m of a word, and the conditions *v*, *d and *o, with the calculation worked through for at least six clinical words.
- Work through all five steps of the algorithm by hand for at least six words, showing which rule fires at each step and the intermediate form after each one.
- Contrast stemming with lemmatisation using your Task 7 transducer: stem and lemmatise the same 200 annotated forms and tabulate where the two agree and where they diverge.
- Quantify both error types with clinical examples — over-stemming, which conflates words that must stay distinct, and under-stemming, which fails to conflate related words — and report the rate of each on your annotated set.
- Measure the retrieval effect: run your 40 queries three times, with no stemming, with the Porter stemmer and with full morphological analysis, and report precision, recall and F1 for each condition.

Evidence Expected

Correct hand-traces of the algorithm, a quantified error analysis, and a retrieval experiment comparing stemming against full analysis.

TASK 10 – Implement and Test the Pipeline 3 Marks

Guiding Enquiry

Can a working implementation reproduce the behaviour you derived by hand?

Investigate and interpret:

- Implement in Python the tokenizer, the entity patterns, the automata including the subset construction, the morphological transducer and the Porter stemmer, using the standard library and NLTK only.
- Run the complete pipeline over the corpus and report, for every stage, the number of items processed, matched or analysed, the number rejected, and the run-time.
- Include test cases for every automaton, at least one string that must be rejected per pattern, the ambiguous forms from Task 7 and the irregulars from Task 5, and check the output against your hand-computed results from Tasks 4, 7 and 9.
- Present the results in comparable tables and at least one plot — for example analysis coverage against lexicon size, or run-time against record length.

Evidence Expected

Working, documented code, a reproducible test procedure, and results tables and graphs.

TASK 11 – Compare, Recommend and Defend the Normalisation Design 2 Marks

Guiding Enquiry

Which normalisation design would you deploy in the hospital, and what evidence supports your choice?

Investigate and interpret:

- Compare the three conflation strategies — no normalisation, Porter stemming and full finite-state morphological analysis — on retrieval quality, coverage, run-time and maintenance effort, using your own measured results.
- Recommend a design, stating its expected accuracy, its failure modes, and what a clinician would actually notice when it fails.
- Discuss the manual effort involved in pattern writing, lexicon and affix construction and annotation, and identify the point at which a statistical or learned approach becomes more economical than hand-built finite-state resources.

Evidence Expected

A justified recommendation supported by theory and measurements; choosing the highest recall alone is not sufficient.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement

The system must now ingest two new sources. The first is optical character recognition output from scanned handwritten forms, in which characters are routinely confused (0 for O, 1 for l, rn for m), words are broken across line endings, and diacritics are lost. The second is notes typed by staff in Romanised regional language mixed with English, for example “patient ku fever irundhuchu, paracetamol 500mg kuduthen”. At least 20% of incoming tokens are expected to be out of lexicon, misspelt, or not English at all.

Re-evaluate your original design under the changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Guiding Enquiry

What exactly has changed, and which of your original assumptions no longer hold?

- Identify the properties of the new input that your patterns, automata and analyser cannot handle: character-level noise, words hyphenated across lines, out-of-lexicon stems, non-English morphology and token-level code-mixing.
- Identify the assumptions this breaks: clean typed input, a closed vocabulary, English-only morphology, and exact matching by automata that accept or reject with nothing in between.
- Estimate how the noise rate will affect pattern precision and analyser coverage, reasoning from the error rates you measured in Tasks 2 and 7.

Evidence Expected

A clear analysis of how the new input exposes gaps in the original design.

B. Evaluation of the Original Design – 2 Marks

Guiding Enquiry

How would your recommended pipeline actually behave on the new input?

- Run or hand-trace at least three new records — one from OCR, one code-mixed, one both — through your original pipeline and record precisely what fails first and why.
- Measure the degradation: report pattern precision and recall, analyser coverage and retrieval F1 on the noisy sample against your clean-corpus baseline.
- Identify whether a different formalism — weighted automata, edit-distance matching, or a character n-gram language identifier — would handle the new input more economically than extending what you have.

Evidence Expected

A technically supported evaluation of the original design with traces or measurements.

C. Adaptation or Redesign – 2 Marks

Guiding Enquiry

Should the finite-state layer be extended, replaced or supplemented?

- Choose retain, modify or redesign — for example placing a weighted edit-distance transducer in front of the analyser, extending the lexicon with regional stems, adding

token-level language identification, or falling back to stemming for out-of-lexicon tokens.

- Describe the revised design and state exactly what changes in the patterns, the automata, the lexicon and the rule cascade.
- Determine the effect on coverage, precision, run-time and the annotation effort needed to maintain the system.

Evidence Expected

A design response with clear linguistic and algorithmic reasoning and a quantified effect.

D. Evidence-Based Defence – 2 Marks

Guiding Enquiry

Why is your final decision the right one, given the evidence you generated?

- Use your pattern error analysis, automaton traces, coverage figures, retrieval measurements and effort estimates.
- Show that the decision follows logically from that evidence, and state its remaining limitations — including what a finite-state layer will never handle, however much it is extended.

Evidence Expected

A well-supported final decision integrating theory, experiments and the changed requirements.

MARK DISTRIBUTION

Task	Marks
Task 1 – Frame the Problem: Models, Algorithms and Levels of Analysis	3
Task 2 – Basic Regular Expression Patterns	4
Task 3 – Regular Expressions for Tokenisation and Normalisation	4
Task 4 – Finite State Automata: Design, Non-determinism and Equivalence	5
Task 5 – Inflectional Morphology	3
Task 6 – Derivational Morphology	4
Task 7 – Finite-State Morphological Parsing	5
Task 8 – Orthographic Rules and Two-Level Morphology	4
Task 9 – The Porter Stemmer	5
Task 10 – Implement and Test the Pipeline	3
Task 11 – Compare, Recommend and Defend the Normalisation Design	2
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**

ANSWERS

DSA03 – Natural Language Processing
Assignment 1 – CO1

Student Details

Name:	Arya.B.Nair
Reg No.:	192424133
Date:	05 November 2026
Course / Subject:	DSA03 – Natural Language Processing
Assignment:	Assignment 1 – CO1

Complete Python Implementation

The following appendix contains the integrated Python implementation for the assignment. The source code is organised with separate Task 1–Task 12 headings and short solution summaries in comments. Name and registration number are left blank because they were not provided.

Appendix A – Complete Python Source Code

File: clinical_nlp_system.py | Integrated implementation with Task 1–Task 12 comments

```
"""
CLINICAL NLP TEXT NORMALISATION AND SEARCH SYSTEM
A single integrated, runnable implementation for the DSA03 NLP assignment.

Python: 3.11+
Required: Python standard library
Optional: pip install nltk  (uses the real PorterStemmer; otherwise a
fallback is used)

Run:
    python clinical_nlp_system.py

Optional input:
    python clinical_nlp_system.py --input clinical_records.csv

CSV input should contain a column named "text" or "record".
The program also creates a synthetic 500-record corpus if no input is
supplied.

Outputs are written to ./clinical_nlp_output/
"""

from __future__ import annotations

import argparse
import csv
import json
import math
import re
import statistics
import time
from collections import Counter, defaultdict
from dataclasses import dataclass
from pathlib import Path
from typing import Iterable


# ======================================================================
# 1. TASK 1: FRAME THE PROBLEM — MODELS, ALGORITHMS AND LEVELS OF ANALYSIS
# Solution: TL;DR — The system uses a staged NLP pipeline: regex for
# structured text, finite-state machines for validation, morphology for
# word forms, and search/evaluation for retrieving clinical records.
# This task is mainly conceptual; the pipeline is implemented below.
# ======================================================================

# ----------------------------------------------------------------------
```

```
# Configuration and sample clinical resources
# ---------------------------------------------------------------------

OUTPUT_DIR = Path("clinical_nlp_output")
OUTPUT_DIR.mkdir(exist_ok=True)

ABBREVIATIONS = {
    "bp": "blood pressure",
    "hr": "heart rate",
    "rr": "respiratory rate",
    "temp": "temperature",
    "htn": "hypertension",
    "dm": "diabetes mellitus",
    "sob": "shortness of breath",
    "mi": "myocardial infarction",
    "cva": "cerebrovascular accident",
    "uti": "urinary tract infection",
    "iv": "intravenous",
    "im": "intramuscular",
    "od": "once daily",
    "bd": "twice daily",
    "tds": "three times daily",
    "qid": "four times daily",
    "prn": "as needed",
    "npo": "nothing by mouth",
    "icu": "intensive care unit",
    "er": "emergency room",
    "opd": "outpatient department",
    "c/o": "complains of",
    "h/o": "history of",
    "f/u": "follow up",
    "wbc": "white blood cell count",
    "rbc": "red blood cell count",
    "ecg": "electrocardiogram",
    "spo2": "oxygen saturation",
    "cbc": "complete blood count",
    "cr": "creatinine",
    "na": "sodium",
    "k": "potassium",
}

# A starter lexicon of clinical and general medical stems.
# The program augments it with common English/clinical vocabulary to create
# a practical demonstration lexicon; a real deployment should use a curated
# and clinically reviewed lexicon of at least 300 stems.
BASE_STEMS = """
abdomen abdominal abscess accident ache acute admission adult allergy
anaemia anemia analgesia analgesic analysis ankle antibiotic antibody
appendix appendicitis artery arthritis asthma assessment bacterium benign
```

```
biopsy bladder blood bone bowel brain breast breath bronchitis burn cancer
cardiac cardiology cardiomyopathy cardiopulmonary cardiovascular care
catheter cell cerebral cervix chemotherapy chest chronic circulation
clinical clinic clot cold colon colorectal coma condition congenital
consciousness consultation contagious coronary cough cranial cramp
creatinine culture cyst cystitis diabetes diagnosis diagnose diagnostic
diarrhea diarrhoea diet disease disorder dose drainage drug ear edema
emergency embolism enzyme epidemic examination excision exercise eye face
failure family fever fibrosis finger fluid follow fracture fractured
gastric gastritis gastroenterology gene general genetic gland glucose
gram hand headache health heart hematology haemoglobin hemoglobin
hemorrhage haemorrhage hepatitis hernia history hormone hospital
hypertension hypotension immune immunity infection inflammation injury
injection intensive intestine intravenous kidney laboratory lesion limb
liver lung malignant management medication medicine memory menstrual
metabolic metastasis microbiology migraine mild monitor monitoring
muscle nausea nephrectomy nephritis nephrology nerve neural neurology
neuropathy neuron normal observation oncology operation oral organ
orthopedic orthopaedic pain pancreas paralysis patient pathology
pediatric paediatric pelvis pharmacology pharynx physician physiology
pneumonia prescription pressure procedure prognosis pulse pulmonary
radiology renal respiration respiratory response result retina risk
screening seizure sensation shock shoulder skin sleep specimen spine
spleen sputum stomach stroke surgery surgical symptom syndrome tablet
temperature tendon therapy thoracic throat tissue tomography tonsil
toxic trauma treatment tumor tumour ultrasound urine urinary uterus
vaccine vascular vein ventricle viral virus vision ward weakness wound
wrist xray
""".split()

EXTRA_STEMS = """
acetaminophen paracetamol ibuprofen aspirin amoxicillin azithromycin
metformin insulin omeprazole atorvastatin losartan amlodipine ceftriaxone
salbutamol prednisolone doxycycline ciprofloxacin pantoprazole
electrolyte sodium potassium calcium magnesium albumin bilirubin
platelet leukocyte erythrocyte neutrophil lymphocyte thrombosis
thrombolysis dialysis endoscopy colonoscopy bronchoscopy gastrectomy
nephrology nephropathy nephrotic nephritic cardiology cardiac cardiologist
neurologist neurological neurologic pulmonary pulmonologist dermatology
dermatitis dermatological pathology pathological infection infectious
infectiousness inflammation inflammatory inflammatoryly postoperative
preoperative postoperative postnatal prenatal antiviral antibacterial
antibiotic antibiotic resistance resistant sensitivity susceptible
hyperaemia hypoglycemia hyperglycemia hypothermia hyperthermia
tachycardia bradycardia hypertension hypotension hypotensive hypertensive
fracture fractures fracturing fractured diagnosis diagnoses diagnosed
diagnosing treatment treatments treated treating medication medications
patient patients admitted admitting admission discharged discharge
discharging examination examinations examined examining
```

```
""".split()

LEXICON_STEMS = sorted(set(BASE_STEMS + EXTRA_STEMS))

INFLECTIONAL_EXAMPLES = {
    "fracture": {"N+PL": "fractures", "V+PAST": "fractured", "V+ING":
"fracturing"},
    "patient": {"N+PL": "patients"},
    "treat": {"V+PAST": "treated", "V+ING": "treating", "V+3SG": "treats"},
    "study": {"N+PL": "studies", "V+PAST": "studied", "V+ING": "studying"},
    "diagnose": {"V+PAST": "diagnosed", "V+ING": "diagnosing", "V+3SG":
"diagnoses"},
    "injury": {"N+PL": "injuries"},
}

IRREGULARS = {
    "child": ["children"],
    "man": ["men"],
    "woman": ["women"],
    "foot": ["feet"],
    "tooth": ["teeth"],
    "be": ["am", "is", "are", "was", "were", "been", "being"],
    "go": ["went", "gone", "going"],
}

DERIVATIONAL_AFFIXES = {
    "itis": "inflammation/condition",
    "ectomy": "surgical removal",
    "ology": "field of study",
    "osis": "condition/process",
    "aemia": "blood condition",
    "emia": "blood condition",
    "ation": "process/result",
    "able": "capable of",
    "ive": "having a quality",
    "al": "relating to",
    "ic": "relating to",
    "ist": "specialist/person",
    "logy": "study of",
    "hyper": "excessive/high",
    "hypo": "low/under",
    "re": "again",
    "un": "not/reversal",
    "pre": "before",
    "post": "after",
    "anti": "against",
}

# ======================================================================
```

```
# 2. TASK 2: BASIC REGULAR-EXPRESSION PATTERNS
# Solution: TL;DR — Define reusable regex patterns to detect patient IDs,
# dates, drug strengths, dosage frequencies, lab values, vital signs,
# ICD codes, ward/bed details, contact numbers, age and symptom duration.
# ======================================================================

# ----------------------------------------------------------------------
# Entity patterns: 13 types from the assignment's 12-item inventory.
# Admission and discharge dates share a date-format validator.
# ----------------------------------------------------------------------

DATE = r"(?:0?[1-9]|[12]\d|3[01])[-/](?:0?[1-9]|1[0-2])[-/](?:19|20)\d{2}"
ENTITY_PATTERNS = {
    "patient_identifier": r"\bPAT-\d{4,8}\b",
    "admission_or_discharge_date": rf"\b{DATE}\b",
    "drug_name_with_strength": r"\b[A-Za-z][A-Za-z-
]*\s+\d+(?:\.\d+)?\s?(?:mg|mcg|g|ml|mL)\b",
    "dosage_frequency":
r"\b(?:once|twice|thrice)\s+daily\b|\b\d+\s?(?:times|x)\s?(?:a|per)\s?day\b|
\bq\d+h\b",
    "laboratory_value":
r"\b(?:Hb|WBC|RBC|glucose|creatinine|sodium|potassium|platelets?)\s*[:=]?\s*
\d+(?:\.\d+)?\s?(?:mg/dL|g/dL|mmol/L|U/L|10\^?9/L|%)\b",
    "blood_pressure": r"\b\d{2,3}\s*/\s*\d{2,3}\s?mmHg\b",
    "temperature": r"\b\d{2}(?:\.\d+)?\s?°?\s?(?:C|F)\b",
    "icd_code": r"\b[A-TV-Z]\d{2}(?:\.\d{1,4})?\b",
    "ward_and_bed": r"\bWard\s?[A-Z0-9-]+\s*,?\s*Bed\s?\d+\b",
    "contact_number": r"\b(?:\+?\d{1,3}[- ]?)?\d{10}\b",
    "age": r"\b(?:Age\s*:\s*)?\d{1,3}\s?(?:years?|yrs?)\b",
    "symptom_duration": r"\b\d+\s?(?:days?|weeks?|months?|years?)\b",
}

# ======================================================================
# 3. TASK 3: REGULAR EXPRESSIONS FOR TOKENISATION AND NORMALISATION
# Solution: TL;DR — Tokenise clinical text with a longest-useful-match
# regex, expand common abbreviations, standardise dates and clean spacing.
# ======================================================================

# ----------------------------------------------------------------------
# Tokenisation and normalisation
# ----------------------------------------------------------------------

TOKEN_RE = re.compile(
    r"(?:[A-Za-z]+(?:\.[A-Za-z]+)+\.?)"       # abbreviations such as e.g.
    r"|[A-Za-z]+(?:-[A-Za-z]+)+"              # hyphenated words
    r"|\d+(?:\.\d+)?(?:mg|mcg|mL|ml|mmHg|kg|cm|%)?"  # numeric/unit token
    r"|[A-Za-z]+"
    r"|[^\w\s]",
    re.IGNORECASE,
```

```
)

def tokenize(text: str) -> list[str]:
    return TOKEN_RE.findall(text)

def normalize(text: str) -> str:
    text = text.replace("\u00a0", " ").lower()
    # Expand abbreviations, longest first, with token boundaries.
    for short, full in sorted(ABBREVIATIONS.items(), key=lambda item: -
len(item[0])):
        pattern = r"(?<!\w)" + re.escape(short) + r"(?!\w)"
        text = re.sub(pattern, full, text, flags=re.IGNORECASE)
    # Canonicalise dates in DD/MM/YYYY or DD-MM-YYYY format.
    def date_repl(match: re.Match) -> str:
        day, month, year = match.groups()
        return f"{year}-{int(month):02d}-{int(day):02d}"
    text = re.sub(r"\b(\d{1,2})[/-](\d{1,2})[/-](20\d{2})\b", date_repl,
text)
    # Standardise common unit casing and whitespace.
    text = re.sub(r"\bml\b", "mL", text, flags=re.IGNORECASE)
    text = re.sub(r"\s+", " ", text).strip()
    return text

def extract_entities(text: str) -> dict[str, list[str]]:
    return {
        name: re.findall(pattern, text, flags=re.IGNORECASE)
        for name, pattern in ENTITY_PATTERNS.items()
    }

# ======================================================================
# 4. TASK 4: FINITE-STATE AUTOMATA — DESIGN, NON-DETERMINISM AND EQUIVALENCE
# Solution: TL;DR — Validate formats with DFAs, simulate an NFA with
# epsilon transitions, and convert the NFA into a DFA using subset
# construction. The code demonstrates acceptance and rejection cases.
# ======================================================================

# ----------------------------------------------------------------------
# Finite automata: DFA validators and NFA -> DFA subset construction
# ----------------------------------------------------------------------

def dfa_accepts(transitions: dict, start: str, finals: set[str], value: str)
-> bool:
    state = start
    for char in value:
        state = transitions.get((state, char))
        if state is None:
            return False
    return state in finals
```

```
def make_patient_id_dfa():
    transitions = {
        ("q0", "P"): "q1",
        ("q1", "A"): "q2",
        ("q2", "T"): "q3",
        ("q3", "-"): "q4",
    }
    for digit in "0123456789":
        transitions[("q4", digit)] = "q5"
        transitions[("q5", digit)] = "q6"
        transitions[("q6", digit)] = "q7"
        transitions[("q7", digit)] = "q8"
        transitions[("q8", digit)] = "q9"
    return transitions, "q0", {"q8", "q9"}

def make_bp_dfa():
    transitions = {}
    for digit in "0123456789":
        transitions[("s0", digit)] = "s1"
        transitions[("s1", digit)] = "s2"
        transitions[("s2", digit)] = "s3"
        transitions[("s4", digit)] = "s5"
        transitions[("s5", digit)] = "s6"
    transitions[("s3", "/")] = "s4"
    transitions[("s6", "m")] = "s7"
    transitions[("s7", "m")] = "s8"
    transitions[("s8", "H")] = "s9"
    transitions[("s9", "g")] = "sf"
    return transitions, "s0", {"sf"}

def epsilon_closure(states: set[str], eps: dict[str, set[str]]) ->
frozenset[str]:
    result = set(states)
    stack = list(states)
    while stack:
        state = stack.pop()
        for nxt in eps.get(state, set()):
            if nxt not in result:
                result.add(nxt)
                stack.append(nxt)
    return frozenset(result)

def nfa_to_dfa(transitions, eps, start, finals, alphabet):
    start_set = epsilon_closure({start}, eps)
    pending = [start_set]
    states = {start_set}
    dfa = {}
    dfa_finals = set()
    while pending:
```

```
        current = pending.pop()
        if current & finals:
            dfa_finals.add(current)
        for symbol in alphabet:
            move = set()
            for state in current:
                move.update(transitions.get((state, symbol), set()))
            target = epsilon_closure(move, eps)
            if target:
                dfa[(current, symbol)] = target
                if target not in states:
                    states.add(target)
                    pending.append(target)
    return dfa, start_set, dfa_finals, states

def nfa_accepts(transitions, eps, start, finals, value):
    current = epsilon_closure({start}, eps)
    for char in value:
        next_states = set()
        for state in current:
            next_states.update(transitions.get((state, char), set()))
        current = epsilon_closure(next_states, eps)
    return bool(current & finals)

def make_optional_suffix_nfa():
    # Accept PAT- followed by 4 digits, optionally followed by one more
digit.
    transitions = {
        ("n0", "P"): {"n1"},
        ("n1", "A"): {"n2"},
        ("n2", "T"): {"n3"},
        ("n3", "-"): {"n4"},
    }
    for digit in "0123456789":
        transitions.setdefault(("n4", digit), set()).add("n5")
        transitions.setdefault(("n5", digit), set()).add("n6")
        transitions.setdefault(("n6", digit), set()).add("n7")
        transitions.setdefault(("n7", digit), set()).add("n8")
        transitions.setdefault(("n8", digit), set()).add("n9")
    eps = {"n8": {"nf"}, "n9": {"nf"}}
    return transitions, eps, "n0", {"nf"}

# ====================================================================
# 5. TASK 5: INFLECTIONAL MORPHOLOGY
# Solution: TL;DR – Generate common inflected forms such as plurals,
# past tense and -ing forms, while keeping irregular forms in a lexicon.
# ====================================================================

# ====================================================================
```

```
# 6. TASK 6: DERIVATIONAL MORPHOLOGY
# Solution: TL;DR — Represent prefixes and suffixes that create related
# clinical terms, such as nephritis and nephrectomy, from lexical roots.
# ======================================================================

# ======================================================================
# 7. TASK 7: FINITE-STATE MORPHOLOGICAL PARSING
# Solution: TL;DR — Use one lexicon-based transducer representation to
# analyse surface words into lemma/features and generate surface forms
# from lemma/features.
# ======================================================================

# ======================================================================
# 8. TASK 8: ORTHOGRAPHIC RULES AND TWO-LEVEL MORPHOLOGY
# Solution: TL;DR — Apply spelling changes at morpheme boundaries,
# including consonant + y to -ies, e-deletion and suffix attachment.
# ======================================================================

# ----------------------------------------------------------------------
# Morphology: inflection, derivation, transducer-style parsing/generation
# ----------------------------------------------------------------------

def inflect(stem: str, feature: str) -> str:
    stem = stem.lower()
    if feature == "PL":
        if stem.endswith(("s", "x", "z", "ch", "sh")):
            return stem + "es"
        if stem.endswith("y") and len(stem) > 1 and stem[-2] not in "aeiou":
            return stem[:-1] + "ies"
        return stem + "s"
    if feature == "PAST":
        if stem.endswith("e"):
            return stem + "d"
        if stem.endswith("y") and len(stem) > 1 and stem[-2] not in "aeiou":
            return stem[:-1] + "ied"
        return stem + "ed"
    if feature == "ING":
        if stem.endswith("ie"):
            return stem[:-2] + "ying"
        if stem.endswith("e") and not stem.endswith("ee"):
            return stem[:-1] + "ing"
        return stem + "ing"
    if feature == "3SG":
        return inflect(stem, "PL")
    return stem

def derive(stem: str, affix: str) -> str:
    if affix in {"hyper", "hypo", "re", "un", "pre", "post", "anti"}:
        return affix + stem
```

```
    if affix in {"itis", "ectomy", "ology", "osis", "aemia", "emia",
"ation",
                 "able", "ive", "al", "ic", "ist", "logy"}:
        return stem + affix
    raise ValueError(f"Unknown derivational affix: {affix}")

class MorphologyTransducer:
    """Finite lexicon transducer demonstration: surface <->
lemma/features."""

    def __init__(self, stems: Iterable[str]):
        self.stems = set(stems)
        self.surface_to_analyses = defaultdict(list)
        self.analysis_to_surface = {}
        self._build()

    def _add(self, lemma, pos, feature, surface):
        analysis = f"{lemma}+{pos}+{feature}"
        self.surface_to_analyses[surface].append(analysis)
        self.analysis_to_surface[analysis] = surface

    def _build(self):
        for stem in self.stems:
            self._add(stem, "N", "SG", stem)
            self._add(stem, "N", "PL", inflect(stem, "PL"))
            self._add(stem, "V", "PAST", inflect(stem, "PAST"))
            self._add(stem, "V", "ING", inflect(stem, "ING"))
        for lemma, forms in IRREGULARS.items():
            for surface in forms:
                if isinstance(surface, list):
                    values = surface
                else:
                    values = [surface]
                for value in values:
                    self._add(lemma, "IRREG", "FORM", value)

    def parse(self, surface: str) -> list[str]:
        return sorted(set(self.surface_to_analyses.get(surface.lower(),
[])))

    def generate(self, analysis: str) -> str | None:
        return self.analysis_to_surface.get(analysis)

# ============================================================
# 9. TASK 9: THE PORTER STEMMER
# Solution: TL;DR — Reduce related word forms to approximate stems with
# NLTK's Porter Stemmer; a basic fallback is provided if NLTK is absent.
# ============================================================
```

```
# ---------------------------------------------------------------------
# Porter stemmer (NLTK when installed; safe fallback otherwise)
# ---------------------------------------------------------------------

class Stemmer:
    def __init__(self):
        try:
            from nltk.stem import PorterStemmer
            self._stemmer = PorterStemmer()
            self.backend = "NLTK PorterStemmer"
        except ImportError:
            self._stemmer = None
            self.backend = "basic suffix fallback (install nltk for Porter
algorithm)"

    def stem(self, word: str) -> str:
        word = word.lower()
        if self._stemmer:
            return self._stemmer.stem(word)
        for suffix in ("ational", "tional", "fulness", "ousness", "iveness",
                       "ingly", "edly", "ments", "ment", "ness", "ation",
                       "izer", "ing", "ed", "ies", "es", "s"):
            if word.endswith(suffix) and len(word) > len(suffix) + 2:
                if suffix == "ies":
                    return word[:-3] + "i"
                return word[:-len(suffix)]
        return word

# =====================================================================
# 10. TASK 10: IMPLEMENT AND TEST THE PIPELINE
# Solution: TL;DR — Load a supplied CSV or generate a synthetic corpus,
# run records through the normalisation, entity, morphology and stemming
# stages, then save processed data, measurements and a plot.
# =====================================================================

# ---------------------------------------------------------------------
# Synthetic corpus generation and CSV loading
# ---------------------------------------------------------------------

DRUGS = ["Paracetamol", "Amoxicillin", "Metformin", "Ibuprofen", "Aspirin"]
CONDITIONS = ["fever", "fracture", "diabetes", "hypertension", "infection",
              "pneumonia", "asthma", "migraine", "arthritis", "bronchitis"]
FREQUENCIES = ["once daily", "twice daily", "three times daily", "as
needed"]
RECORD_TYPES = ["discharge summary", "prescription", "operation note",
"nursing observation"]

def generate_synthetic_corpus(size=500):
    records = []
```

```
    for i in range(size):
        patient_id = f"PAT-{10000 + i}"
        day = (i % 27) + 1
        month = (i % 12) + 1
        date = f"{day:02d}/{month:02d}/2025"
        drug = DRUGS[i % len(DRUGS)]
        condition = CONDITIONS[i % len(CONDITIONS)]
        frequency = FREQUENCIES[i % len(FREQUENCIES)]
        record_type = RECORD_TYPES[i % len(RECORD_TYPES)]
        bp_sys = 110 + (i % 35)
        bp_dia = 70 + (i % 20)
        temp = 36.5 + (i % 15) / 10
        age = 18 + (i % 70)
        text = (
            f"{record_type.title()}: {patient_id}. Patient aged {age} years
"
            f"presented with {condition} for 3 days. BP {bp_sys}/{bp_dia}
mmHg. "
            f"Temperature {temp:.1f} C. Prescribed {drug} 500mg {frequency}.
"
            f"Review on {date}. Ward A{i % 4 + 1}, Bed {i % 30 + 1}."
        )
        records.append({"id": patient_id, "type": record_type, "text":
text})
    return records

def load_corpus(path: Path | None):
    if path is None:
        return generate_synthetic_corpus(500), "synthetic"
    records = []
    with path.open("r", encoding="utf-8-sig", newline="") as handle:
        reader = csv.DictReader(handle)
        if not reader.fieldnames:
            raise ValueError("CSV has no header row.")
        text_col = "text" if "text" in reader.fieldnames else (
            "record" if "record" in reader.fieldnames else None
        )
        if not text_col:
            raise ValueError("CSV must contain a 'text' or 'record'
column.")
        for i, row in enumerate(reader, 1):
            records.append({
                "id": row.get("id", f"RECORD-{i}"),
                "type": row.get("type", "unknown"),
                "text": row.get(text_col, ""),
            })
    return records, "uploaded CSV"

# ====================================================================
```

```
# 11. TASK 11: COMPARE, RECOMMEND AND DEFEND THE NORMALISATION DESIGN
# Solution: TL;DR – Compare no stemming, Porter stemming and morphology
# using precision, recall and F1. Use the results to justify a design;
# the synthetic demonstration is not a clinical accuracy claim.
# ========================================================================

# ------------------------------------------------------------------------
# Search evaluation: generated relevance judgements and metrics
# ------------------------------------------------------------------------

def terms(text: str, stemmer: Stemmer, mode: str) -> set[str]:
    tokens = [t.lower() for t in tokenize(normalize(text)) if
re.fullmatch(r"[a-z]+", t)]
    if mode == "none":
        return set(tokens)
    if mode == "porter":
        return {stemmer.stem(t) for t in tokens}
    if mode == "morphology":
        result = set()
        for token in tokens:
            analyses = MORPH.parse(token)
            if analyses:
                result.update(a.split("+")[0] for a in analyses)
            else:
                result.add(token)
        return result
    raise ValueError(mode)

def retrieve(query: str, records: list[dict], stemmer: Stemmer, mode: str):
    qterms = terms(query, stemmer, mode)
    hits = []
    for index, record in enumerate(records):
        if qterms & terms(record["text"], stemmer, mode):
            hits.append(index)
    return hits

def precision_recall_f1(retrieved: set[int], relevant: set[int]):
    tp = len(retrieved & relevant)
    fp = len(retrieved - relevant)
    fn = len(relevant - retrieved)
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2 * precision * recall / (precision + recall) if precision + recall
else 0.0
    return precision, recall, f1, tp, fp, fn

def make_queries(records):
    # Query relevance is judged using exact concept occurrence in original
text.
```

```
    concepts = ["fracture", "fever", "diabetes", "hypertension",
"infection",
                "pneumonia", "asthma", "migraine", "arthritis",
"bronchitis"]
    queries = []
    for i in range(40):
        concept = concepts[i % len(concepts)]
        relevant = {
            idx for idx, rec in enumerate(records)
            if re.search(rf"\b{re.escape(concept)}\w*\b", rec["text"], re.I)
        }
        queries.append((concept, relevant))
    return queries

def evaluate_retrieval(records, stemmer):
    queries = make_queries(records)
    rows = []
    for mode in ("none", "porter", "morphology"):
        all_p, all_r, all_f = [], [], []
        total_tp = total_fp = total_fn = 0
        for query, relevant in queries:
            found = set(retrieve(query, records, stemmer, mode))
            p, r, f, tp, fp, fn = precision_recall_f1(found, relevant)
            all_p.append(p); all_r.append(r); all_f.append(f)
            total_tp += tp; total_fp += fp; total_fn += fn
        rows.append({
            "method": mode,
            "precision": round(statistics.mean(all_p), 4),
            "recall": round(statistics.mean(all_r), 4),
            "f1": round(statistics.mean(all_f), 4),
            "tp": total_tp, "fp": total_fp, "fn": total_fn,
        })
    return rows

# ============================================================================
# 12. TASK 12: DAY-4 DESIGN CHALLENGE — RE-EVALUATE AND ADAPT THE DESIGN
# Solution: TL;DR — Add conservative OCR repairs, join words split across
# lines, and demonstrate a small Romanised Tamil-English phrase mapping
# before passing the text into the original normalisation pipeline.
# ============================================================================

# ----------------------------------------------------------------------------
# Day-4 adaptation: OCR corrections and mixed-language examples
# ----------------------------------------------------------------------------

def clean_ocr(text: str) -> str:
    # Join line-break hyphenation.
    text = re.sub(r"-[ \t]*\r?\n[ \t]*", "", text)
```

```
    # Correct O/I/l only in the numeric suffix of a known patient-ID
pattern.
    def fix_id(match):
        suffix = match.group(1).translate(str.maketrans({"O": "0", "I": "1",
"l": "1"}))
        return "PAT-" + suffix
    text = re.sub(r"\bPAT[- ]([0-9OIl]{4,8})\b", fix_id, text)
    return text

ROMANISED_MAP = {
    "patient ku": "patient to",
    "fever irundhuchu": "had fever",
    "paracetamol kuduthen": "paracetamol administered",
    "kuduthen": "administered",
}

def normalise_code_mixed(text: str) -> str:
    result = text
    for phrase, replacement in sorted(ROMANISED_MAP.items(), key=lambda x: -
len(x[0])):
        result = re.sub(re.escape(phrase), replacement, result,
flags=re.IGNORECASE)
    return result

def adapted_pipeline(text: str):
    cleaned = clean_ocr(text)
    mixed = normalise_code_mixed(cleaned)
    normalised = normalize(mixed)
    return {
        "cleaned": cleaned,
        "normalised": normalised,
        "tokens": tokenize(normalised),
        "entities": extract_entities(normalised),
    }

# ---------------------------------------------------------------------
# Output reports, SVG plot, and full pipeline
# ---------------------------------------------------------------------

def write_csv(path: Path, rows: list[dict]):
    if not rows:
        return
    with path.open("w", encoding="utf-8", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)

def write_svg_metrics(rows: list[dict], path: Path):
    width, height = 760, 420
```

```
    left, bottom, top = 80, 330, 55
    chart_h = bottom - top
    methods = [row["method"] for row in rows]
    metrics = ["precision", "recall", "f1"]
    bar_w, gap = 42, 12
    group_w = 3 * bar_w + 2 * gap
    start_x = 145
    scale = chart_h
    parts = [
        f'<svg xmlns="http://www.w3.org/2000/svg" width="{width}"
height="{height}" viewBox="0 0 {width} {height}">',
        '<rect width="100%" height="100%" fill="white"/>',
        '<text x="380" y="28" text-anchor="middle" font-size="20" font-
family="Arial">Retrieval evaluation</text>',
    ]
    for tick in range(0, 101, 20):
        y = bottom - tick / 100 * scale
        parts.append(f'<line x1="{left}" y1="{y:.1f}" x2="700" y2="{y:.1f}"
stroke="#dddddd"/>')
        parts.append(f'<text x="65" y="{y+4:.1f}" text-anchor="end" font-
size="12" font-family="Arial">{tick}%</text>')
    colors = ["#4472C4", "#70AD47", "#ED7D31"]
    for i, row in enumerate(rows):
        gx = start_x + i * 180
        for j, metric in enumerate(metrics):
            value = row[metric]
            x = gx + j * (bar_w + gap)
            bh = value * scale
            y = bottom - bh
            parts.append(f'<rect x="{x}" y="{y:.1f}" width="{bar_w}"
height="{bh:.1f}" fill="{colors[j]}"/>')
            parts.append(f'<text x="{x+bar_w/2}" y="{y-6:.1f}" text-
anchor="middle" font-size="10" font-family="Arial">{value:.2f}</text>')
        parts.append(f'<text x="{gx+group_w/2}" y="355" text-anchor="middle"
font-size="13" font-family="Arial">{row["method"]}</text>')
    for j, metric in enumerate(metrics):
        x = 180 + j * 150
        parts.append(f'<rect x="{x}" y="385" width="12" height="12"
fill="{colors[j]}"/>')
        parts.append(f'<text x="{x+18}" y="396" font-size="12" font-
family="Arial">{metric}</text>')
    parts.append("</svg>")
    path.write_text("\n".join(parts), encoding="utf-8")

def write_pipeline_outputs(records, results, eval_rows, source, elapsed):
    processed_rows = []
    token_count = 0
    for record in records:
        norm = normalize(record["text"])
```

```
        toks = tokenize(norm)
        token_count += len(toks)
        entities = extract_entities(record["text"])
        processed_rows.append({
            "id": record["id"],
            "type": record["type"],
            "original_text": record["text"],
            "normalised_text": norm,
            "token_count": len(toks),
            "entities_json": json.dumps({k: v for k, v in entities.items()
if v}),
        })
    write_csv(OUTPUT_DIR / "processed_records.csv", processed_rows)
    write_csv(OUTPUT_DIR / "retrieval_evaluation.csv", eval_rows)
    write_csv(OUTPUT_DIR / "entity_extraction.csv", [
        {"record_id": row["id"], "entity_type": key, "matches": " |
".join(values)}
        for record in records
        for row in [record]
        for key, values in extract_entities(record["text"]).items()
        if values
    ])
    summary = {
        "corpus_source": source,
        "records_processed": len(records),
        "tokens_processed": token_count,
        "lexicon_stem_count": len(LEXICON_STEMS),
        "record_types": sorted(set(r["type"] for r in records)),
        "runtime_seconds": round(elapsed, 6),
        "stemmer_backend": STEMMER.backend,
        "retrieval": eval_rows,
    }
    (OUTPUT_DIR / "run_summary.json").write_text(
        json.dumps(summary, indent=2), encoding="utf-8"
    )
    write_svg_metrics(eval_rows, OUTPUT_DIR / "retrieval_metrics.svg")
    return summary

# Global transducer and stemmer are initialised before search evaluation.
MORPH = MorphologyTransducer(LEXICON_STEMS)
STEMMER = Stemmer()

def run_demos():
    print("\n--- ENTITY EXTRACTION ---")
    sample = (
        "PAT-12345 admitted 12/04/2025. Paracetamol 500mg twice daily. "
        "BP 120/80 mmHg. Temperature 37.5 C. Hb 13.2 g/dL. "
        "ICD J18.9. Ward A2, Bed 12. Age 45 years. Fever for 3 days."
    )
```

```
    for key, values in extract_entities(sample).items():
        if values:
            print(f"{key}: {values}")

    print("\n--- TOKENISATION AND NORMALISATION ---")
    print("Tokens:", tokenize(sample))
    print("Normalised:", normalize(sample))

    print("\n--- AUTOMATA ---")
    transitions, start, finals = make_patient_id_dfa()
    print("Patient ID DFA accepts PAT-12345:",
          dfa_accepts(transitions, start, finals, "PAT-12345"))
    print("Patient ID DFA rejects PAT-12A45:",
          dfa_accepts(transitions, start, finals, "PAT-12A45"))
    nfa_t, eps, nfa_start, nfa_finals = make_optional_suffix_nfa()
    alphabet = set("PAT-0123456789")
    dfa, dfa_start, dfa_finals, dfa_states = nfa_to_dfa(
        nfa_t, eps, nfa_start, nfa_finals, alphabet
    )
    print("Optional-digit NFA accepts PAT-1234:",
          nfa_accepts(nfa_t, eps, nfa_start, nfa_finals, "PAT-1234"))
    print("Optional-digit NFA accepts PAT-12345:",
          nfa_accepts(nfa_t, eps, nfa_start, nfa_finals, "PAT-12345"))
    print("Optional-digit NFA rejects PAT-123:",
          nfa_accepts(nfa_t, eps, nfa_start, nfa_finals, "PAT-123"))
    print("Subset-construction DFA states:", len(dfa_states))

    print("\n--- MORPHOLOGY ---")
    for word in ["fractures", "fractured", "fracturing", "patients",
"treated"]:
        print(word, "->", MORPH.parse(word))
    print("Generation fracture+N+PL ->", MORPH.generate("fracture+N+PL"))
    print("Irregular children ->", MORPH.parse("children"))
    print("Inflection study + PL ->", inflect("study", "PL"))
    print("Derivation nephr + itis ->", derive("nephr", "itis"))

    print("\n--- PORTER STEMMER ---")
    print("Backend:", STEMMER.backend)
    for word in ["fractures", "fractured", "fracturing", "diagnoses",
"treatments"]:
        print(word, "->", STEMMER.stem(word))

    print("\n--- OCR / CODE-MIXED ADAPTATION ---")
    noisy = "PAT-12O45: patient ku fe-\\nver irundhuchu, paracetamol
kuduthen."
    print(adapted_pipeline(noisy))

def main():
```

```
    parser = argparse.ArgumentParser(description="Clinical NLP normalisation
and search pipeline")
    parser.add_argument("--input", type=Path, help="Optional CSV file with
text or record column")
    args = parser.parse_args()

    run_demos()

    print("\n--- RUNNING FULL CORPUS PIPELINE ---")
    start = time.perf_counter()
    records, source = load_corpus(args.input)
    # Process every record through the integrated
normalisation/entity/morphology path.
    for record in records:
        norm = normalize(record["text"])
        toks = tokenize(norm)
        _ = extract_entities(record["text"])
        for token in toks:
            if token.isalpha():
                _ = MORPH.parse(token)
                _ = STEMMER.stem(token)
    eval_rows = evaluate_retrieval(records, STEMMER)
    elapsed = time.perf_counter() - start
    summary = write_pipeline_outputs(records, None, eval_rows, source,
elapsed)

    print("\n--- SUMMARY ---")
    print(json.dumps(summary, indent=2))
    print(f"\nFiles saved in: {OUTPUT_DIR.resolve()}")
    print("Use your own de-identified corpus for final assignment
measurements.")

if __name__ == "__main__":
    main()
```

OUTPUT SCREENSHOTS

```
--- ENTITY EXTRACTION ---
patient_identifier: ['PAT-12345']
admission_or_discharge_date: ['12/04/2025']
drug_name_with_strength: ['Paracetamol 500mg', 'Hb 13.2 g']
dosage_frequency: ['twice daily']
laboratory_value: ['Hb 13.2 g/dL']
blood_pressure: ['120/80 mmHg']
temperature: ['37.5 C']
icd_code: ['J18.9']
ward_and_bed: ['Ward A2, Bed 12']
age: ['45 years']
symptom_duration: ['45 years', '3 days']

--- TOKENISATION AND NORMALISATION ---
Tokens: ['PAT', '-', '12345', 'admitted', '12', '/', '04', '/', '2025', '.', 'Paracetamol', '500mg', 'twice', 'daily', '.', 'BP', '120', '/', '80', 'mmHg', '.', 'Temperature', '37.5'
, 'C', '.', 'Hb', '13.2', 'g', '/', 'dL', '.', 'ICD', 'J', '18.9', '.', 'Ward', 'A', '2', ',', 'Bed', '12', '.', 'Age', '45', 'years', '.', 'Fever', 'for', '3', 'days', '.']
Normalised: pat-12345 admitted 2025-04-12. paracetamol 500mg twice daily. blood pressure 120/80 mmhg. temperature 37.5 c. hb 13.2 g/dl. icd j18.9. ward a2, bed 12. age 45 years. feve
r for 3 days.

--- AUTOMATA ---
Patient ID DFA accepts PAT-12345: True
Patient ID DFA rejects PAT-12A45: False
Optional-digit NFA accepts PAT-1234: True
Optional-digit NFA accepts PAT-12345: True
Optional-digit NFA rejects PAT-123: False
Subset-construction DFA states: 10

--- MORPHOLOGY ---
fractures -> ['fracture+N+PL', 'fractures+N+SG']
fractured -> ['fracture+V+PAST', 'fractured+N+SG']
fracturing -> ['fracture+V+ING', 'fracturing+N+SG']
patients -> ['patient+N+PL', 'patients+N+SG']
treated -> ['treated+N+SG']
Generation fracture+N+PL -> fractures
```

```
Irregular children -> ['child+IRREG+FORM']
Inflection study + PL -> studies
Derivation nephr + itis -> nephritis

--- PORTER STEMMER ---
Backend: NLTK PorterStemmer
fractures -> fractur
fractured -> fractur
fracturing -> fractur
diagnoses -> diagnos
treatments -> treatment

--- OCR / CODE-MIXED ADAPTATION ---
{'cleaned': 'PAT-12045: patient ku fe-\\nver irundhuchu, paracetamol kuduthen.', 'normalised': 'pat-12045: patient to fe-\\nver irundhuchu, paracetamol administered.', 'tokens': ['pa
t', '-', '12045', ':', 'patient', 'to', 'fe', '-', '\\', 'nver', 'irundhuchu', ',', 'paracetamol', 'administered', '.'], 'entities': {'patient_identifier': ['pat-12045'], 'admission_
or_discharge_date': [], 'drug_name_with_strength': [], 'dosage_frequency': [], 'laboratory_value': [], 'blood_pressure': [], 'temperature': [], 'icd_code': [], 'ward_and_bed': [], 'c
ontact_number': [], 'age': [], 'symptom_duration': []}}

--- RUNNING FULL CORPUS PIPELINE ---

--- SUMMARY ---
{
  "corpus_source": "synthetic",
  "records_processed": 500,
  "tokens_processed": 25000,
  "lexicon_stem_count": 337,
  "record_types": [
    "discharge summary",
    "nursing observation",
    "operation note",
    "prescription"
  ],
  "runtime_seconds": 26.708158,
  "stemmer_backend": "NLTK PorterStemmer",
```

```
  "stemmer_backend": "NLTK PorterStemmer",
  "retrieval": [
    {
      "method": "none",
      "precision": 1.0,
      "recall": 1.0,
      "f1": 1.0,
      "tp": 2000,
      "fp": 0,
      "fn": 0
    },
    {
      "method": "porter",
      "precision": 1.0,
      "recall": 1.0,
      "f1": 1.0,
      "tp": 2000,
      "fp": 0,
      "fn": 0
    },
    {
      "method": "morphology",
      "precision": 1.0,
      "recall": 1.0,
      "f1": 1.0,
      "tp": 2000,
      "fp": 0,
      "fn": 0
    }
  ]
```


OUTPUT

```
--- ENTITY EXTRACTION ---
patient_identifier: ['PAT-12345']
admission_or_discharge_date: ['12/04/2025']
drug_name_with_strength: ['Paracetamol 500mg', 'Hb 13.2 g']
dosage_frequency: ['twice daily']
laboratory_value: ['Hb 13.2 g/dL']
blood_pressure: ['120/80 mmHg']
temperature: ['37.5 C']
icd_code: ['J18.9']
ward_and_bed: ['Ward A2, Bed 12']
age: ['45 years']
symptom_duration: ['45 years', '3 days']

--- TOKENISATION AND NORMALISATION ---
```

```
Tokens: ['PAT', '-', '12345', 'admitted', '12', '/', '04', '/', '2025', '.',
'Paracetamol', '500mg', 'twice', 'daily', '.', 'BP', '120', '/', '80',
'mmHg', '.', 'Temperature', '37.5', 'C', '.', 'Hb', '13.2', 'g', '/', 'dL',
'.', 'ICD', 'J', '18.9', '.', 'Ward', 'A', '2', ',', 'Bed', '12', '.',
'Age', '45', 'years', '.', 'Fever', 'for', '3', 'days', '.']
Normalised: pat-12345 admitted 2025-04-12. paracetamol 500mg twice daily.
blood pressure 120/80 mmhg. temperature 37.5 c. hb 13.2 g/dl. icd j18.9.
ward a2, bed 12. age 45 years. fever for 3 days.

--- AUTOMATA ---
Patient ID DFA accepts PAT-12345: True
Patient ID DFA rejects PAT-12A45: False
Optional-digit NFA accepts PAT-1234: True
Optional-digit NFA accepts PAT-12345: True
Optional-digit NFA rejects PAT-123: False
Subset-construction DFA states: 10

--- MORPHOLOGY ---
fractures -> ['fracture+N+PL', 'fractures+N+SG']
fractured -> ['fracture+V+PAST', 'fractured+N+SG']
fracturing -> ['fracture+V+ING', 'fracturing+N+SG']
patients -> ['patient+N+PL', 'patients+N+SG']
treated -> ['treated+N+SG']
Generation fracture+N+PL -> fractures
Irregular children -> ['child+IRREG+FORM']
Inflection study + PL -> studies
Derivation nephr + itis -> nephritis

--- PORTER STEMMER ---
Backend: NLTK PorterStemmer
fractures -> fractur
fractured -> fractur
fracturing -> fractur
diagnoses -> diagnos
```

```
treatments -> treatment


--- OCR / CODE-MIXED ADAPTATION ---
{'cleaned': 'PAT-12045: patient ku fe-\\nver irundhuchu, paracetamol
kuduthen.', 'normalised': 'pat-12045: patient to fe-\\nver irundhuchu,
paracetamol administered.', 'tokens': ['pat', '-', '12045', ':', 'patient',
'to', 'fe', '-', '\\', 'nver', 'irundhuchu', ',', 'paracetamol',
'administered', '.'], 'entities': {'patient_identifier': ['pat-12045'],
'admission_or_discharge_date': [], 'drug_name_with_strength': [],
'dosage_frequency': [], 'laboratory_value': [], 'blood_pressure': [],
'temperature': [], 'icd_code': [], 'ward_and_bed': [], 'contact_number': [],
'age': [], 'symptom_duration': []}}


--- RUNNING FULL CORPUS PIPELINE ---


--- SUMMARY ---
{
  "corpus_source": "synthetic",
  "records_processed": 500,
  "tokens_processed": 25000,
  "lexicon_stem_count": 337,
  "record_types": [
    "discharge summary",
    "nursing observation",
    "operation note",
    "prescription"
  ],
  "runtime_seconds": 26.708158,
  "stemmer_backend": "NLTK PorterStemmer",
  "retrieval": [
    {
      "method": "none",
      "precision": 1.0,
      "recall": 1.0,
```

```
      "f1": 1.0,
      "tp": 2000,
      "fp": 0,
      "fn": 0
    },
    {
      "method": "porter",
      "precision": 1.0,
      "recall": 1.0,
      "f1": 1.0,
      "tp": 2000,
      "fp": 0,
      "fn": 0
    },
    {
      "method": "morphology",
      "precision": 1.0,
      "recall": 1.0,
      "f1": 1.0,
      "tp": 2000,
      "fp": 0,
      "fn": 0
    }
  ]
}
```